

DAB multi-environment deployment

Core idea

Build the application once. Deploy the same tested artifact with controlled environment-specific configuration.

Databricks Declarative Automation Bundles (DABs), formerly called Databricks Asset Bundles, define and deploy Databricks application resources as code. A bundle can contain jobs, pipelines, notebooks, Python files, Python wheel artifacts, permissions, and target-specific deployment configuration.

The application code and resource definitions should remain common. Targets such as **dev**, **qual**, and **prod** provide only the values and controls that genuinely differ.

```
One Git commit
-> one tested Python wheel
-> one DAB resource definition
    -> dev configuration -> dev workspace
    -> qual configuration -> qual workspace
    -> prod configuration -> prod workspace
```

This avoids manually rebuilding jobs in the Databricks UI and prevents three environment definitions from drifting apart.

DAB is not the complete infrastructure layer

DAB and Terraform have different responsibilities.

Layer	Responsibility
Terraform	Azure resource groups, Databricks workspaces, identities, catalogs, schemas, grants, budgets, and other long-lived foundations
DAB	Databricks application jobs, pipelines, wheel deployment, workload permissions, and target overlays
GitHub Actions	Testing, building, approvals, target selection, artifact promotion, and deployment orchestration
GitHub environments	Environment-specific workspace and identity identifiers plus approval rules
Unity Catalog	Governed data access for catalogs, schemas, tables, volumes, functions, and models

Terraform prepares the destination. DAB deploys the application into that destination.

Repository structure

```
azure-deployment-w-terraform/
├── databricks.yml
├── resources/
│   ├── retail_pipeline.yml
│   └── customer_cdc.pipeline.yml
├── src/
│   └── retail_analytics/
├── pyproject.toml
├── .github/workflows/
│   └── deploy.yml
├── infrastructure/
│   └── stacks/
```

The files have separate roles:

- **databricks.yml** defines the bundle, wheel build, variables, targets, workspace deployment paths, naming prefixes, and runtime identity.
- **resources/** contains the shared jobs and pipelines.

- **src/** contains the Python application packaged into the wheel.
- **pyproject.toml** defines the Python package, version, dependencies, and entry points.
- **deploy.yml** selects the GitHub environment, obtains OIDC authentication, validates the bundle, and deploys it.
- **infrastructure/** contains the Terraform foundations.

What databricks.yml does

The root configuration connects the shared application to its deployment targets.

```
bundle:
  name: retail-analytics-smoke-test

include:
  - resources/*.yml

artifacts:
  wheel:
    type: whl
    build: uv build --wheel
```

This tells DAB to:

1. Use a stable bundle identity.
2. include the job and pipeline definitions under resources/.
3. build the Python project as a wheel before deployment.

The project declares variables without committing environment values:

```
variables:
  deployment_service_principal_application_id:
    description: Identity that manages bundle resources.
  catalog:
    description: Unity Catalog catalog used by the workload.
  runtime_service_principal_application_id:
    description: Identity used when the job runs.
```

Resource files consume those values using DAB substitutions:

```
permissions:
  - service_principal_name: ${var.deployment_service_principal_application_id}
    level: CAN_MANAGE

run_as:
  service_principal_name: ${var.runtime_service_principal_application_id}

parameters:
  - --catalog
  - ${var.catalog}
```

Targets and workspace separation

The same bundle defines three targets:

```
targets:
  dev:
    default: true
    presets:
      name_prefix: "[dev]"

  qual:
    presets:
      name_prefix: "[qual]"

  prod:
    presets:
      name_prefix: "[prod]"
```

The selected target changes the deployment context without copying the application.

Each target also uses an isolated workspace path:

```
/Workspace/Applications/retail-analytics-smoke-test/dev/.bundle/...  
/Workspace/Applications/retail-analytics-smoke-test/qual/.bundle/...  
/Workspace/Applications/retail-analytics-smoke-test/prod/.bundle/...
```

When the environments use separate Databricks workspaces, the path is isolated again by the workspace boundary.

What changes between environments

Configuration	dev	qual	prod
Workspace	Development workspace	Qualification workspace	Production workspace
Catalog	Development catalog	Qualification catalog	Production catalog
Deployment identity	Dev deployer	Qual deployer	Prod deployer
Runtime identity	Dev runtime identity	Qual runtime identity	Prod runtime identity
OIDC policy	Exact dev subject and audience	Exact qual subject and audience	Exact prod subject and audience
Compute	Minimal or serverless	Production-like validation	Approved production policy
Schedule	Manual or paused	Manual or controlled	Approved production schedule
Approval	Automatic after main CI	Manual protected promotion	Manual protected promotion
Terraform state	Dev state key	Qual state key	Prod state key

Application source, tests, task dependencies, and wheel contents should not change during promotion.

GitHub environment configuration

GitHub contains protected environments named **dev**, **qual**, and **prod**. Each environment defines the same variable names with different values:

Variable	Purpose
DATABRICKS_HOST	Selects the destination Databricks workspace
DATABRICKS_CLIENT_ID	Selects the deployment service principal
DATABRICKS_RUNTIME_CLIENT_ID	Selects the job runtime service principal
DATABRICKS_CATALOG	Selects the Unity Catalog catalog

The workflow maps these values into DAB:

```
DATABRICKS_CLIENT_ID  
-> DAB deployment identity  
-> owner or manager of bundle resources  
  
DATABRICKS_RUNTIME_CLIENT_ID  
-> DAB run_as identity  
-> identity used when a job executes  
  
DATABRICKS_CATALOG  
-> DAB catalog variable  
-> catalog passed to the Python wheel
```

These values are identifiers, not passwords. Authentication requires GitHub's signed, short-lived OIDC token and a matching Databricks federation policy.

No Databricks PAT, OAuth client secret, or human password is required by the deployment workflow.

CI/CD deployment flow

```
Pull request
-> build dev container
-> lint and unit tests
-> required review
-> squash and merge

Main branch
-> CI succeeds
-> dev GitHub environment selected
-> GitHub issues OIDC token
-> Databricks validates federation policy
-> wheel is built
-> bundle is validated
-> bundle is deployed to dev
-> deployment ends without running the job

Qual or prod
-> manual workflow dispatch
-> protected GitHub environment approval
-> environment-specific OIDC identity
-> validate and deploy selected target
```

The core commands are:

```
databricks bundle validate --strict --target dev
databricks bundle deploy --target dev

databricks bundle validate --strict --target qual
databricks bundle deploy --target qual

databricks bundle validate --strict --target prod
databricks bundle deploy --target prod
```

Deploying a bundle creates or updates definitions and uploads files. It does not run the job unless the workflow explicitly calls a run command.

Current compute model: serverless

The current retail smoke job uses serverless task environments:

```
tasks:
- task_key: bronze
  environment_key: default
  python_wheel_task:
    package_name: retail_analytics
    entry_point: retail-pipeline

environments:
- environment_key: default
  spec:
    environment_version: "4"
    dependencies:
      - ../dist/*.whl
```

With serverless compute:

- Databricks manages the infrastructure.
- The bundle does not select a VM node type.
- The bundle does not set a Spark runtime version.
- The bundle does not set worker counts.
- Compute starts only when the job runs, not when it is deployed.
- This is appropriate for the current low-cost deployment prototype.

Different serverless environments can still use different workspace governance, catalogs, identities, schedules, parameters, budgets, and supported serverless controls.

Future classic-compute model

Classic job compute is appropriate when a workload requires explicit node types, runtime versions, autoscaling ranges, libraries, networking, or cluster policies.

The shared task definition would reference one job cluster:

```
tasks:
- task_key: bronze
  job_cluster_key: shared_job_cluster
  python_wheel_task:
    package_name: retail_analytics
    entry_point: retail-pipeline
```

Each target can then override only the compute definition:

```
targets:
  dev:
    resources:
      jobs:
        retail_medallion_smoke_test:
          job_clusters:
            - job_cluster_key: shared_job_cluster
              new_cluster:
                policy_id: <dev-policy-id>
                node_type_id: <small-dev-node>
                autoscale:
                  min_workers: 1
                  max_workers: 2

  qual:
    resources:
      jobs:
        retail_medallion_smoke_test:
          job_clusters:
            - job_cluster_key: shared_job_cluster
              new_cluster:
                policy_id: <qual-policy-id>
                node_type_id: <qual-node>
                autoscale:
                  min_workers: 1
                  max_workers: 4

  prod:
    resources:
      jobs:
        retail_medallion_smoke_test:
          job_clusters:
            - job_cluster_key: shared_job_cluster
              new_cluster:
                policy_id: <prod-policy-id>
                node_type_id: <prod-node>
                autoscale:
                  min_workers: 2
                  max_workers: 8
```

A cluster policy should restrict approved runtimes, node families, autoscaling limits, tags, security mode, and cost controls. DAB selects the appropriate policy; it should not bypass platform governance.

A task cannot simultaneously use serverless environment_key and classic job_cluster_key. Switching one target to classic compute is therefore a deliberate structural variation. Validate every target and avoid duplicating complete task graphs.

Identity separation

Two service principals have different jobs:

Identity	Responsibility	Should not normally have
Deployment service principal	Authenticate from GitHub, upload the wheel, create or update jobs, reconcile bundle permissions	Business-data write access
Runtime service principal	Execute the job and access only approved Unity Catalog objects	Deployment ownership or broad workspace administration

Use separate deployment and runtime identities per environment. A dev identity must not be trusted to deploy to prod.

```
GitHub dev -> dev federation policy -> dev deployer -> dev workspace
GitHub qual -> qual federation policy -> qual deployer -> qual workspace
GitHub prod -> prod federation policy -> prod deployer -> prod workspace
```

Build once and promote

The desired artifact path is:

```
commit
  -> tested wheel and checksum
  -> dev deployment
  -> qual promotion
  -> prod promotion
```

Production should receive the exact wheel checksum that passed CI and earlier validation. Rebuilding separately for each environment can produce a different artifact even when the source commit is unchanged.

The current repository builds and records the dev wheel. Reusing that exact wheel during later qual and prod promotion remains a planned enhancement.

Do we need dev.json, qual.json, and prod.json?

Not for the current deployment configuration.

Use DAB targets for workspace resources and target overrides. Use GitHub environments for workspace and identity identifiers. Add JSON only when the Python application needs substantial runtime data such as source mappings, table rules, CDC behavior, or validation thresholds.

```
DAB YAML          -> deployment and resource configuration
GitHub environments -> destination and identity configuration
Optional JSON     -> large application runtime configuration
Secret manager    -> credentials and sensitive values
```

Never commit tokens, passwords, client secrets, storage keys, or local authentication files.

Final design

```
Terraform
  -> provisions dev, qual, and prod foundations
  -> keeps independent state and access boundaries

DAB
  -> builds and deploys one application definition
  -> selects dev, qual, or prod target
  -> applies controlled target overrides

GitHub Actions
  -> validates code and bundle
  -> deploys dev automatically
  -> protects qual and prod with manual approval
  -> authenticates using OIDC

Unity Catalog
  -> governs what the runtime identity can read and write
```

The result is one traceable application release deployed consistently across isolated environments, with infrastructure, deployment, runtime, and data-governance responsibilities kept separate.

New workspace checklist

For every additional environment:

1. Provision the workspace and Unity Catalog foundations with Terraform.
2. Use an independent Terraform state key and access boundary.
3. Create environment-specific deployment and runtime service principals.
4. Grant workspace access and bundle-resource ownership to the deployer.
5. Apply least-privilege Unity Catalog grants to the runtime identity.
6. Create the exact GitHub OIDC federation policy for the environment.
7. Configure the matching protected GitHub environment variables.
8. Require reviewers for qual and prod.
9. Validate and deploy without running the job.
10. Execute a separately approved smoke test.

For local validation, use an explicit CLI profile for each workspace:

```
databricks bundle validate --strict --target dev --profile dev
databricks bundle validate --strict --target qual --profile qual
databricks bundle validate --strict --target prod --profile prod
```

Never rely on an implicit profile for production. Confirm the target, profile, host, Azure subscription, and planned changes before deployment.

Implementation status

Capability	State
One bundle with dev, qual, and prod targets	Implemented
GitHub environment selects host, identities, and catalog	Implemented
Automatic dev deployment	Implemented
Manual qual and prod targets	Implemented
Environment-specific classic-compute pattern	Documented; not active
Separate qual workspace and federation policy	TBD
Separate prod workspace and federation policy	TBD
Independent Terraform state per additional workspace	TBD
Promote the identical dev wheel to qual and prod	TBD
Environment-specific smoke tests and rollback runbook	TBD